City Brain Math Core — Reasoning Document

TSYP14 City Brain Convergence — Deterministic/Mathematical Decision Layer

Prepared for readers who have not read the code: judges, and teammates deciding whether to trust this component.

---

1. What this component is, and what it is not

City Brain reads a partner "Twin" system's live sensor stream, detects incidents, decides a response, and dispatches an actuator — but only through a chain of other components that check trust, verify safety, and manage the incident's lifecycle. This document covers one link in that chain: the **deterministic, mathematical decision layer**. It is a small, portable, general-purpose software library — not tied to this project's vocabulary at all — that answers four narrow, well-defined questions with deterministic math:

1. Given several pieces of evidence, each with a trust weight, what is the combined probability that something real is happening?
2. Given a stream of numeric readings, has it shown a sustained shift, as opposed to ordinary noise?
3. Given that two kinds of events are correlated, how much should one adjust the odds of the other — and can that adjustment ever be driven to an extreme?
4. Given a probability and a table of costs, what is the cheapest action — and given several incidents and few response units, which unit should go where?

What it explicitly does not do is read raw sensor data, verify a cryptographic signature, decide whether a sensor can be trusted, track an incident's lifecycle, call an actuator, or run the language model that advises a human operator. Those are other components, built by other people, and nothing in this document should be read as a claim about them. This component's job is narrower and, because it is narrower, it can be proven correct in a way a larger, more entangled system could not be.

Why this scoping matters

A system that reacts to signed sensor data from a partner it has never met must assume that partner's sensors — or the partner's software itself — can be made to lie, even while the lies carry a perfectly valid cryptographic signature. A signature proves who sent a message. It says nothing about whether the message is true. The central engineering problem this component solves is: **given that any single sensor reading might be fabricated, how does a system combine many readings into a decision without letting one fabricated reading decide anything by itself?**

---

2. Each algorithm, explained conceptually

2.1 Evidence fusion with a capped influence

Imagine ten witnesses reporting on whether a fire is happening. Each witness is more or less convincing — one just says "I smell smoke," another says "I saw flames." A natural way to combine many such reports is **Bayesian reasoning in log-odds**: convert the starting belief (the "prior") and each witness's testimony into log-odds, add them up, and convert back to a probability. This is mathematically the correct way to combine independent pieces of evidence, and it is simple enough to compute deterministically and audit by hand.

The problem: if one witness is allowed to say "I am ten billion times more certain than anyone could possibly be," their single opinion can swamp everyone else's and single-handedly flip the final verdict. The fix used here is a **cap**: no single piece of evidence, however extreme its claim, is allowed to contribute more than a fixed, configured amount to the final answer. A witness screaming with infinite confidence is heard exactly as loudly as a witness who is simply very sure — no louder.

Crucially, the cap applies not just to each individual report but to each independent *source* of reports — so an attacker who floods the system with a thousand fabricated readings from one compromised device gains nothing beyond what *one* fabricated reading from that device would have gained. The cap is on the source, not on the message count.

Why not something simpler, like a fixed threshold on one reading? A fixed threshold ("if water level > X, it's a flood") cannot combine multiple kinds of evidence, has no notion of how trustworthy each piece is, and offers no way to reason about how many compromised sources it would take to fool it. The capped log-odds approach gives a single number — "how many independent sources does it take" — and that number is provable, not just plausible.

2.2 Change-point detection: EWMA + CUSUM

Many real quantities (water level, traffic density) drift slowly before crossing any meaningful threshold. A system that only checks "is the current reading above X?" will miss a slow, sustained rise that stays just under X for a long time, then suddenly matters. The fix is to track not just the current reading but an **adaptive running average** (an exponentially weighted moving average, EWMA) that learns the normal level and spread of each stream on its own, and a **cumulative sum** (CUSUM) that accumulates small, persistent deviations from that average over time. A single noisy blip does not accumulate meaningfully; a real, sustained shift does, and eventually crosses an alarm threshold even though no single reading would have.

Why not a fixed threshold? Because this library has no prior knowledge of what "normal" looks like for an arbitrary stream from an unfamiliar partner system — the EWMA learns that from the data itself, which is what lets the same code work against a Twin nobody has seen before.

An important boundary. This detector answers "has this already-trusted stream shown a sustained shift in the real-world quantity it measures" — it does **not** attempt to detect whether the sensor itself has been tampered with, has failed, or is lying. That is a different, harder problem that belongs to a separate part of the system (a trust/security layer), and this component deliberately does not try to re-solve it or second-guess it. Keeping that boundary sharp is itself a safety property: two components each half-solving the same problem is worse than one component solving its half well and trusting the other to solve the rest.

2.3 The correlation graph: evidence that events are related, kept bounded

Some incidents make others more or less likely: an accident often precedes a traffic jam nearby; a flood and a fire reported in the same place at the same moment is unusual. A small graph captures this, with each connection carrying a multiplier that either raises or lowers the odds of a related event.

The critical design decision: a multiplier that *lowers* the odds of some other event is never allowed to drive those odds all the way to zero, no matter how many such lowering connections are chained together. This matters because an attacker who can fake one kind of event (say, a flood) must never be able to use that fake to fully silence a real report of something else (a fire) nearby. The library enforces a floor — a minimum possible multiplier — and proves, not just claims, that no chain of suppressing connections, however long, can push the combined effect below that floor.

2.4 Expected-cost decision and optimal assignment

Once a probability is known, deciding what to do is a matter of weighing costs: a false alarm costs something (wasted resources, public trust); a missed real incident costs something else, usually much more. Given those costs explicitly as numbers, the cheapest action is a simple, auditable calculation — no hidden judgment calls, and the exact break-even probability can be read off directly.

When several incidents compete for a limited number of response units, assigning them one-by-one by whichever seems most urgent first (a "greedy" approach) is not guaranteed to be the cheapest overall assignment, and worse, it can give a different answer depending on what order the incidents happen to arrive in — which makes a logged decision hard to defend after the fact. The **Hungarian algorithm** computes the mathematically optimal assignment directly, and it does so in time proportional to a small power of the number of incidents and units — fast enough to run in real time, and its answer does not depend on the arrival order.

2.5 Behavior under the adversarial scenarios this was built for

- **A single falsified sensor reading, however extreme its claim**: bounded to the cap, provably cannot cross a well-chosen action threshold alone.
- **A frozen or dead sensor**: outside the scope of this component (see §2.2); handled by a separate trust/security layer, by design.
- **A slow drift engineered to stay under a naive threshold**: this is exactly what the CUSUM accumulator is for, and it is demonstrably more sensitive to a sustained small drift than a naive fixed-threshold rule, while still correctly staying silent on ordinary noise.
- **Several falsified sources acting together**: the exact number of independent sources required to cross a threshold is a number this library computes in advance, and the fusion math is checked to reach — and never exceed — exactly that number, not approximately.
- **An attempt to suppress a real report by faking a "contradicting" event**: the correlation graph's floor guarantees the suppression can shrink real evidence's effective odds only by a bounded, known amount, never past a hard floor — so strong real evidence always still gets through.

2.6 An optional add-on: grounding evidence in real physics

Everything above treats each piece of evidence's "likelihood ratio" — how strongly a reading supports an incident — as a number that simply arrives from somewhere. A later, optional addition to this component lets that number come from an actual physical model instead of a guess, for the three incident types where a well-established one exists:

- **Flood**: a textbook reservoir model (the same accounting used across hydrology: water in minus water out equals how fast the level rises) predicts how fast a zone's water level should be rising given an inflow and the zone's drainage. A sensor's observed rate of rise is compared against that prediction.
- **Fire**: a published fire-engineering growth curve (how fast heat output grows in a fire's first minutes, with four standard "speed classes" — slow, medium, fast, ultra-fast — used throughout fire safety codes) combined with a standard model of how smoke spreads downwind, predicts what a nearby air-quality sensor should see.
- **Traffic**: a classic 1935 traffic-engineering relationship between how crowded a road is and how fast cars can move predicts the expected speed at a given congestion level, which a sensor's observed speed is compared against.

In every case the comparison between "what physics predicts" and "what the sensor reported" produces the same kind of likelihood-ratio number Section 2.1 already described — capped the same way, fed into the same fusion calculation, with no special treatment and no exemption from the fault-tolerance guarantee below. This add-on changes nothing about how evidence is combined; it only offers a more principled way to produce one of the numbers being combined. It is optional, and the deterministic core works identically with or without it.

The same scope boundary as the change-point detector applies here too: this only ever asks "how consistent is this reading with a real incident," never "should this reading be believed." It does not and cannot flag a reading as fake.

---

3. The fault-tolerance inequality

3.1 In plain language, first

Picture the system's belief as a dial that starts at some baseline position (the "prior" — how likely this kind of event is before any evidence arrives) and gets nudged by each independent piece of evidence. Because every piece of evidence has a capped maximum nudge, moving the dial from its starting position all the way to the "act now" position takes a *minimum number* of independent nudges — and that minimum number is something we can calculate exactly, in advance, for any choice of cap, starting belief, and action threshold.

This is the mathematical heart of the safety guarantee: **one compromised, maximally-fabricated source, by itself, is never enough** — as long as the configured cap and threshold are such that the minimum required number of sources is more than one, which is a property we can check and prove for any specific configuration before it is deployed.

3.2 The formula

Let:

- `p0` = the starting belief (prior probability), strictly between 0 and 1
- `c` = the cap on how extreme a single piece of evidence can be (as a likelihood-ratio multiplier), strictly greater than 1
- `T` = the action threshold (probability), strictly between 0 and 1
- `n` = the number of independent, maximally-capped sources acting together

Convert probabilities to "log-odds" (a standard transformation: `logit(p) = ln(p / (1-p))`). The largest possible combined belief after `n` capped sources is:

```
L_max(n) = logit(p0) + n × ln(c)
p_max(n) = 1 / (1 + e^(-L_max(n)))
```

The smallest number of sources needed to reach the threshold is:

```
n_min = the smallest whole number n such that p_max(n) ≥ T
```

which works out to:

```
n_min = ceiling of [ (logit(T) − logit(p0)) / ln(c) ]
```

This is the number that matters: if an attacker (or a single fault) can control strictly fewer than `n_min` independent evidence sources, the system is mathematically guaranteed not to act on that evidence alone, regardless of how extreme the fabricated readings claim to be.

3.3 Worked example, matching this project's own illustrative figures

With a cap of 8× (`c = 8`), a starting belief of 2% (`p0 = 0.02`), and an action threshold of 20% (`T = 0.20`):

- One maximally fabricated source pushes the belief to **about 14%** — still below the 20% threshold.
- **Two** independent sources are required to cross it; with two, the belief reaches about **57%**.

So `n_min = 2` for these numbers. This was checked two ways — once by hand with the formula above, and once by running the actual compiled code — and both agree exactly, to the last digit a computer can represent. This cross-check is not a formality: during development it caught and helped fix a subtle internal inconsistency in an early version of the code (a boundary case where two different parts of the program could, in principle, have disagreed about whether exactly the minimum number of sources was "enough" — fixed by making every part of the program answer that question the same way, once, rather than twice).

---

4. Calibration bounds: why slow poisoning does not work

This system is allowed to adjust its own thresholds over time, based on confirmed outcomes — a reasonable thing for any adaptive system to do. The risk is a **slow poisoning attack**: an adversary who cannot force a sudden, obviously-wrong decision might instead try to nudge a learned threshold a tiny amount at a time, patiently, over many rounds, until it has drifted somewhere dangerous.

Two simple limits close this off entirely:

1. **Every single adjustment is capped** to a small maximum step size, no matter how large the signal pushing for a bigger change appears to be.
2. **The adjusted value can never leave a hard, pre-configured safe range**, regardless of how many adjustments accumulate.

Together, these mean that moving a threshold any meaningful distance requires a *minimum number* of accepted adjustments — a number this library can calculate in advance — which gives whoever reviews the system's logs a concrete, known amount of time (measured in "how many genuine, confirmed-outcome updates would this take") to notice a sustained one-directional drift before it reaches anywhere consequential. A single bad update, however extreme, moves the threshold by at most one small step.

---

5. Grounded in prior work, not ad hoc invention

Every technique in this component is an application of an established, published idea, not something invented for this project:

- **Change-point detection** (the EWMA-adapted baseline plus the CUSUM accumulator) follows the classical statistical theory of detecting a change in a stream of measurements, dating back to the 1950s and standardized in the reference textbook on the subject. More recent work specifically on detecting faults and attacks in physical/sensor systems confirms that a detector which accumulates evidence over time catches gradual shifts that a detector looking at only the current reading misses — which is exactly why this library pairs the two rather than relying on a single snapshot test.
- **Capped, bounded-influence evidence combination** sits inside a broader, well-studied question in the statistics and signal-processing literature: how many corrupted or compromised sources can a system that combines evidence from many sources tolerate before its ability to tell truth from fabrication collapses entirely? That literature (on "Byzantine-resilient" distributed detection) proves that, past a critical fraction of compromised sources, some systems can be driven completely blind. This component sidesteps that failure mode by construction — because every single source's influence is capped *before* anything is combined, there is no configuration-dependent tipping point to find; the worst case for any given number of compromised sources is always the exact, computable number from §3, not a collapse.
- **Optimal assignment** (the Hungarian algorithm) and **expected-cost decision rules** are textbook results in operations research and statistical decision theory, in use since the 1950s, applied here without modification beyond making the tie-breaking behavior deterministic and reproducible — a practical necessity for a system whose decisions may need to be replayed and audited later, not a change to the underlying mathematics.

---

6. What was actually verified

This section lists what was tested, with real counts, so the reader can see evidence this was tested, not just designed and asserted.

Adversarial scenarios, each a named, explicit test against an adversary modeled as a fully compromised caller handing maximum trust weight to fabricated evidence:

- A single fabricated source at full trust weight, checked across 96 combinations of cap, starting belief, and threshold: **zero threshold crossings**.
- Coordinated fabrication across multiple independent sources, checked for exactness (one source short of the computed minimum never crosses; exactly the computed minimum always does) across 135 configurations, 134 of which were in the tested range and **all 134 confirmed exact**.
- A thousand fabricated readings funneled through one compromised source: confirmed to have exactly the same effect as a single reading from that source — no more.
- An attempted suppression attack (faking one kind of event specifically to mute a real report of another): confirmed that strong, genuine, corroborating evidence still crosses the action threshold even under the worst suppression the system's configured floor allows.
- Hostile or malformed trust weights (above the valid range, negative, or not a real number): all rejected with a defined error rather than silently accepted or clamped.

Randomized property testing, which checks that the mathematical guarantees above hold not just for one worked example but across a very large number of randomly generated configurations within the library's documented valid ranges:

- The default automated run samples **10,000 random configurations per property**, across seven distinct properties (the fault-tolerance bound's exactness, the evidence ceiling, that grouping evidence never helps an attacker, that more evidence never lowers a belief it supports, that the correlation graph's floor holds, that calibration updates stay within bounds, and that the decision rule agrees with its own computed break-even point), plus a separate check of optimal-assignment correctness — **over 1.07 million individual checks from this property-testing pass alone, zero failures** (the project's full test suite, including every hand-written unit and adversarial test, runs over 1.29 million checks in total).
- An extended run samples **50,000 configurations per property** — over **5.39 million property-testing checks, zero failures** — and is available as an on-demand, longer-running verification pass.
- The Python-language version of this library (a separate, independently-written binding layer, not a copy of the C code) was checked against **12,150 additional randomized examples**, again zero failures, confirming the guarantee holds identically regardless of which language is calling into the underlying compiled code.

Golden reference vectors: sixty-seven hand-picked test cases — including this document's own worked example and seventeen covering the physics add-on — were computed completely independently, in a separate script using arbitrary-precision arithmetic that does not call this library's own code at all, specifically so that the test is not circular. Both the C implementation and the Python bindings were checked against this independent reference, to a numerical tolerance of one part in a trillion, and the headline result (one fabricated source reaching 14.035...%) was found to agree to every digit a computer can represent.

Honest limitations, stated plainly: the measured rate of false alarms from the change-point detector, under the specific setting that keeps a real sustained shift visible (rather than being gradually absorbed into the learned baseline), is measurably higher than a simplified textbook formula alone would predict — about 11-12 false alarms per 2000 readings in testing, versus roughly 4 predicted by the simplified theory. This was measured directly rather than assumed, and is exactly why no single detector alarm is ever treated as conclusive on its own — consistent with this project's broader design, where no single signal of any kind is allowed to trigger an action by itself. Separately, the physics add-on's comparison tool (Section 2.6) has a known blind spot: for a sensor reading so extreme it overshoots even the hypothesized incident's own prediction, the comparison can mistakenly read as strong support for the incident rather than recognizing the reading as implausible either way. This is demonstrated on purpose in the test suite rather than hidden, and does not weaken the fault-tolerance guarantee itself, since every piece of evidence — however it was produced — is still subject to the same cap described in Section 3.

Verified independently: before this document's conclusions were finalized, a separate reviewer with no involvement in building this component was asked to check it from scratch against the original project brief — rebuilding it, running every test themselves, and tracing sample calculations through the code by hand rather than taking any claim on faith. That review found no blocking problems and confirmed the central guarantee is genuinely proven and genuinely tested, not merely asserted.

---

7. Closing summary

This component is small by design: four narrow mathematical tools, each doing one well-defined job, each provably bounded against the specific kind of attack it exists to resist, and each checked not only by hand-worked examples but by tens of thousands of randomized configurations and a second, independent implementation in a different language. (An optional fifth piece, Section 2.6, lets one of those tools draw on real physics instead of a guess when producing its input — it changes nothing about the tools themselves or their guarantees.) It makes no claim about the physical world, about whether any particular sensor should be trusted, or about anything happening outside the pure mathematics of combining bounded evidence into a bounded decision. That narrowness is what makes the central claim — *one fabricated source cannot, by itself, trigger an action* — something that can be proven rather than merely hoped for.